

ปัญญาประดิษฐ์ยุคปัจจุบัน

สัปดาห์ที่ 8: LLM ทำงานอย่างไร

ผศ. ดร. อธิพล ฟองแก้ว
ittipon@g.sut.ac.th

วันนี้

ครั้งแรกของวิชานี้เราสร้าง "ตัวแทนอัจฉริยะ" ด้วยการค้นหา ตรรกะ และความน่าจะเป็น ครึ่งหลังเราจะสร้างมันด้วย โมเดล ภาษาขนาดใหญ่ (Large Language Model)

ชั่วโมงที่ 1

จากข้อความสู่ตัวเลข

- โครงข่ายประสาทฉบับย่อ
- การแบ่งโทเคน
- เวกเตอร์ฝังตัว
- การเข้ารหัสตำแหน่ง

ชั่วโมงที่ 2

กลไกความสนใจ

- Query, Key, Value
- ตัวอย่างคำนวณเต็มรูปแบบ
- หน้ากากเชิงสาเหตุ
- บล็อกทรานส์ฟอร์มเมอร์

ชั่วโมงที่ 3

จากทรานส์ฟอร์มเมอร์สู่ LLM

- การทำนายโทเคนถัดไป
- การสุ่มและอุณหภูมิ
- วงจรการฝึก
- ข้อจำกัดและภูมิทัศน์

กติกาของสัปดาห์นี้

สิ่งที่เราจะทำ

- คำนวณด้วยมือจริง ๆ ทุกสูตรหลัก
- มีโจทย์ระหว่างเรียน 5 ข้อ พร้อมเฉลยทันที
- ปิดท้ายด้วยการรันโค้ดจริงในแล็บ

สิ่งที่เราจะไม่ทำ

- ไม่ฝึกโมเดลจากศูนย์ (ไม่มีเวลาและไม่จำเป็น)
- ไม่ท่องสถาปัตยกรรมทุกตัวที่มีในโลก

คำถามที่ต้องตอบได้เมื่อจบคาบ

1. ทำไมภาษาไทยถึงแพงกว่าภาษาอังกฤษเมื่อเรียก API
2. ทำไมบริบทยาวถึงแพงแบบกำลังสอง
3. อุณหภูมิเท่าไรสำหรับงานสกัดข้อมูล และเพราะอะไร
4. ทำไมโมเดลถึงหลง และมันแก้ที่ตัวโมเดลได้ไหม

ชั่วโมงที่ 1

จากข้อความสู่ตัวเลข

เพอร์เซปตรอน: หน่วยพื้นฐาน

หน่วยประมวลผลหนึ่งตัวรับเวกเตอร์เข้า คูณน้ำหนัก บวกไบแอส แล้วผ่านฟังก์ชันกระตุ้น

$$y = \sigma \left(\sum_{i=1}^n w_i x_i + b \right) = \sigma(\mathbf{w}^T \mathbf{x} + b)$$

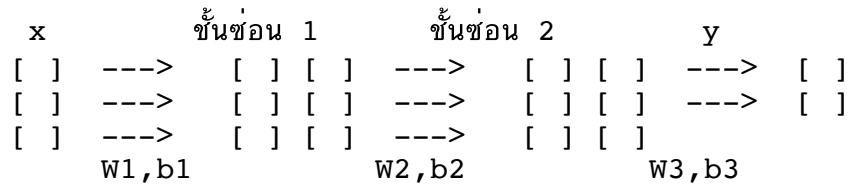
- $\mathbf{x}$ คือข้อมูลเข้า, $\mathbf{w}$ คือน้ำหนักที่เรียนรู้ได้, b คือไบแอส
- σ คือฟังก์ชันไม่เชิงเส้น เช่น ReLU: $\sigma(z) = \max(0, z)$

ทำไมต้องไม่เชิงเส้น? ถ้าซ่อนชั้นเชิงเส้นล้วน ๆ ผลลัพธ์ก็ยังเป็นเชิงเส้น

$$W_2(W_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = \underbrace{(W_2 W_1)}_{\mathbf{W}'} \mathbf{x} + \underbrace{(W_2 \mathbf{b}_1 + \mathbf{b}_2)}_{\mathbf{b}'}$$

การซ่อนชั้นจะไม่เพิ่มพลังการแทนค่าเลยแม้แต่น้อย

โครงข่ายหลายชั้น (Multi-layer Perceptron)



$$\mathbf{h}_1 = \sigma(W_1 \mathbf{x} + \mathbf{b}_1), \quad \mathbf{h}_2 = \sigma(W_2 \mathbf{h}_1 + \mathbf{b}_2), \quad \mathbf{y} = W_3 \mathbf{h}_2 + \mathbf{b}_3$$

ทฤษฎีบทการประมาณสากล: โครงข่ายที่มีชั้นซ่อนเพียงชั้นเดียวและหน่วยมากพอ สามารถประมาณฟังก์ชันต่อเนื่องใด ๆ ได้ใกล้เคียงเท่าที่ต้องการ

แต่ทฤษฎีบทไม่ได้บอกว่าต้องใช้กี่หน่วย และไม่ได้บอกว่าจะหาจำนวนหน่วยนั้นเจอได้อย่างไร ความลึกจึงสำคัญในทางปฏิบัติ แม้ความกว้างจะเพียงพอในทางทฤษฎี

การเรียนรู้ = การหาค่าต่ำสุดของฟังก์ชันสูญเสีย

1. วัดความผิด ด้วยฟังก์ชันสูญเสีย $L(\theta)$
2. หาความชัน $\nabla_{\theta} L$ ด้วยการแพร่กระจายย้อนกลับ (backpropagation)
3. ปรับพารามิเตอร์ ตามทิศทางที่ลดความผิด

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta)$$

การแพร่กระจายย้อนกลับคือกฎลูกโซ่ของแคลคูลัสที่จัดระเบียบให้คำนวณซ้ำน้อยที่สุด

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial \mathbf{h}_2} \cdot \frac{\partial \mathbf{h}_2}{\partial \mathbf{h}_1} \cdot \frac{\partial \mathbf{h}_1}{\partial W_1}$$

สำหรับโมเดลภาษา ฟังก์ชันสูญเสียคืออะไร

โมเดลทำนายการแจกแจงเหนือคลังศัพท์ทั้งหมด แล้ววัดด้วย **cross-entropy**

$$L = -\frac{1}{T} \sum_{t=1}^T \log P(x_t^{\text{จริง}} | x_{<t})$$

อ่านเป็นภาษาคน: "โมเดลให้ความน่าจะเป็นกับโทเคนที่เกิดขึ้นจริงเท่าไร" ถ้าให้ความน่าจะเป็นสูง **log** ใกล้ 0 ค่าสูญเสียต่ำ ถ้าให้ต่ำ ค่าสูญเสียพุ่งขึ้น

Perplexity คือค่าที่รายงานกันบ่อยกว่า เป็นแค่ **L** ที่แปลงกลับเป็นสเกลที่ตีความได้

$$\text{PPL} = e^L$$

ตีความ PPL: "โดยเฉลี่ย โมเดลลังเลอยู่ระหว่างกี่ตัวเลือก"

PPL = 1 คือทำนายถูกเป๊ะทุกครั้ง · PPL = 50000 คือเดามั่วจากคลังศัพท์ทั้งหมด
โมเดลปัจจุบันอยู่ราว 3 ถึง 10 บนข้อความภาษาอังกฤษทั่วไป

ตัวอย่าง: ทั้งหมดนี้เขียนได้ใน PyTorch ไม่กี่บรรทัด

```
import torch, torch.nn as nn

model = nn.Sequential(
    nn.Linear(3, 64), nn.ReLU(),
    nn.Linear(64, 64), nn.ReLU(),
    nn.Linear(64, 1),
)
loss_fn = nn.MSELoss()
opt = torch.optim.Adam(model.parameters(), lr=1e-3)

for x, y in dataloader:
    pred = model(x)
    loss = loss_fn(pred, y)
    opt.zero_grad()      # ล้างความชันเก่า
    loss.backward()      # แพร่กระจายย้อนกลับ (autograd ทำให้ง่าย)
    opt.step()           # อัปเดตพารามิเตอร์
```

ทุกโมเดลภาษาที่เราจะใช้ในอีก 6 สัปดาห์ ฝึกด้วยรูปหน้าต่างแบบนี้ แค่นี้ก็ใหญ่กว่ามาก

ปัญหาแรก: โมเดลไม่เข้าใจตัวอักษร

โครงข่ายประสาทรับได้แต่ตัวเลข เราต้องแปลงข้อความเป็นเวกเตอร์

ขั้นที่ 1: การแบ่งโทเคน (Tokenization) แบ่งข้อความเป็นหน่วยย่อย

"ปัญหาประดิษฐ์ทำงานอย่างไร"

--> ["ปัญห", "ญา", "ประ", "ดิษ", "ฐ", "ทำงาน", "อย่าง", "ไร"]

--> [15234, 8901, 4412, 77120, 3345, 2201, 990, 1187]

ทางเลือกที่เป็นไปได้ 3 แบบ

วิธี	ขนาดคลังศัพท์	ปัญหา
ตัวอักษรเดี่ยว	เล็กมาก (~100)	ลำดับยาวมาก, attention $O(n^2)$ ระเบิด
คำเต็ม	ใหญ่มาก (~1M)	เจอคำใหม่แล้วตาย (out-of-vocabulary)
หน่วยย่อยของคำ (subword) พอดี (32k-256k) ทางที่ทุกคนใช้จริง		

Byte-Pair Encoding: อักขรทีมทั้งหมด

1. เริ่มจากหน่วยเล็กที่สุด (ไบต์เดียว)
2. นับความถี่ของ "คู่ที่อยู่ติดกัน" ทั้งหมด
3. รวมคู่ที่พบบ่อยที่สุดเป็นโทเคนใหม่ 1 ตัว
4. ทำซ้ำข้อ 2-3 จนได้จำนวนโทเคนตามที่กำหนด

เหตุผลที่ทำงานบนไบต์ ไม่ใช่ตัวอักษร: ไบต์มีแค่ 256 ค่า จึงครอบคลุมทุกภาษา ทุกอีโมจิ ทุกอักขระในโลก โดยไม่มีทางเจอ "ตัวอักษรที่ไม่รู้จัก" เลย

ผลข้างเคียงที่สำคัญมาก: อักษรไทย 1 ตัว ใช้ 3 ไบต์ ใน UTF-8 ขณะที่อักษรละติน 1 ตัว ใช้ 1 ไบต์ คลังศัพท์ของ โมเดลส่วนใหญ่ฝึกจากข้อความอังกฤษเป็นหลัก คู่ไบต์ภาษาไทยจึงถูกรวมน้อยกว่า

แบบฝึกหัด 1.1: รัน BPE ด้วยมือ

คลังข้อความมี 4 คำ พร้อมความถี่

low	x 5
lower	x 2
newest	x 6
widest	x 3

เริ่มจากแยกเป็นตัวอักษร แล้วเติม `</w>` ท้ายคำ เช่น `l o w </w>`

จงหา merge 3 ครั้งแรก พร้อมแสดงความถี่ของคู่ที่ชนะในแต่ละรอบ

เฉลย 1.1 (รอบที่ 1)

นับคู่ที่ติดกันทั้งหมด ถ่วงด้วยความถี่ของคำ

คู่	มาจาก	ความถี่
e s	newest(6), widest(3)	9
s t	newest(6), widest(3)	9
l o	low(5), lower(2)	7
o w	low(5), lower(2)	7
t </w>	newest(6), widest(3)	9
n e	newest(6)	6

มีหลายคู่ได้ 9 เท่ากัน ให้เลือกตามลำดับที่พบก่อน ได้ merge 1: e s → es

low </w>	x5	l o w </w>
newest	x6	n e w e s t </w>
widest	x3	w i d e s t </w>

เฉลย 1.1 (รอบที่ 2 และ 3)

รอบที่ 2: ตอนนี้ `es t` มีความถี่ 9 (สูงสุด) ได้ **merge 2: `es t` → `est`**

```
newest  x6  n e w est </w>
widest  x3  w i d est </w>
```

รอบที่ 3: `est </w>` มีความถี่ 9 ได้ **merge 3: `est </w>` → `est</w>`**

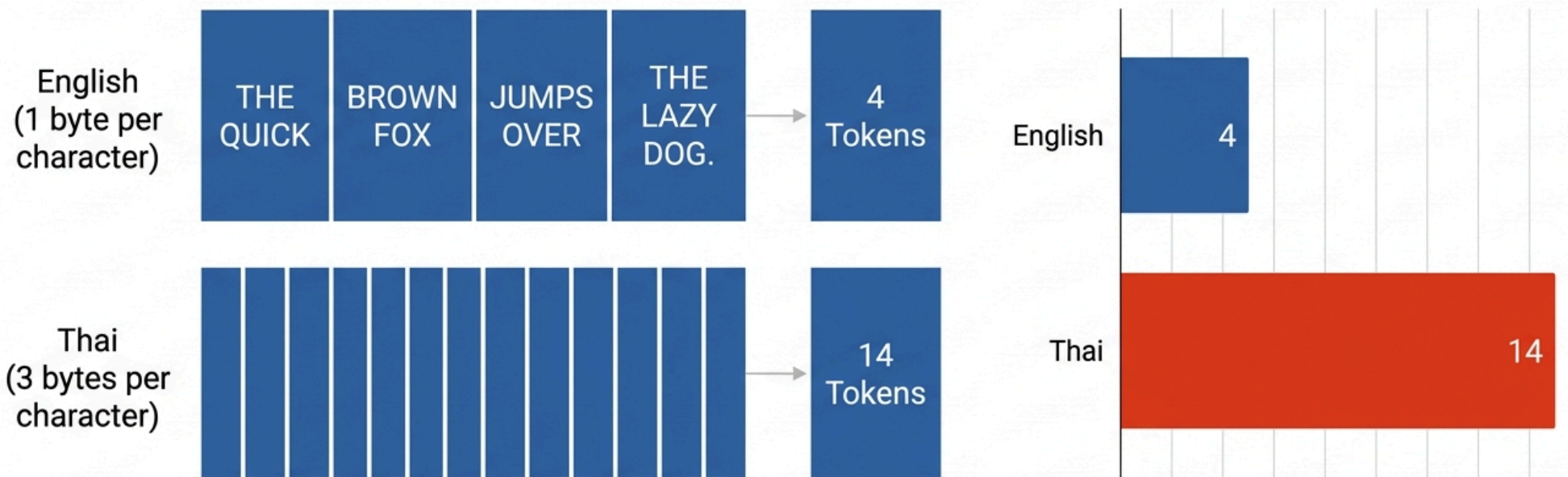
```
newest  x6  n e w est</w>
widest  x3  w i d est</w>
```

สังเกตสิ่งที่เกิดขึ้น: BPE ค้นพบหน่วยคำ `-est` (suffix ขั้นสุด) ได้เอง โดยไม่มีใครสอนไวยากรณ์ให้เลย นี่คือเหตุผลที่มันได้ผลข้ามภาษา

ตรวจสอบ: จำนวนโทเคนรวมลดลงจาก 63 เหลือ 45 ทุก merge ต้องลดจำนวนโทเคนเสมอ

ต้นทุนที่แท้จริงของภาษาไทย

Why Thai costs more tokens



byte-level BPE merges Latin pairs far more often,
so Thai needs 2-4x more tokens for the same meaning

แบบฝึกหัด 1.2: คำนวณค่าใช้จ่ายจริง

สมมติคุณสร้างแชทบอตตอบคำถามนักศึกษา และมีข้อมูลดังนี้

- ราคาโมเดล: input 0.15 USD ต่อ 1M โทเคน, output 0.60 USD ต่อ 1M โทเคน
- พรอมป์ระบบภาษาไทย 500 โทเคน (ส่งทุกครั้ง)
- คำถามเฉลี่ย 80 โทเคน, คำตอบเฉลี่ย 300 โทเคน
- มีคำถาม 2,000 ครั้งต่อวัน

ก) ค่าใช้จ่ายต่อวันเท่าไร

ข) ถ้าเขียนพรอมป์ระบบเป็นภาษาอังกฤษแทน เหลือ 180 โทเคน จะประหยัดได้กี่เปอร์เซ็นต์ของค่า input

เฉลย 1.2

ก) ค่าใช้จ่ายต่อวัน

$$\text{input} = (500 + 80) \times 2000 = 1,160,000 \text{ โทเคน} \times \frac{0.15}{10^6} = 0.174 \text{ USD}$$

$$\text{output} = 300 \times 2000 = 600,000 \text{ โทเคน} \times \frac{0.60}{10^6} = 0.360 \text{ USD}$$

รวม $\approx 0.534 \text{ USD}$ ต่อวัน $\approx 195 \text{ USD}$ ต่อปี

ข) เปลี่ยนพรมป้ตระบบเป็นอังกฤษ

$$\text{input ใหม่} = (180 + 80) \times 2000 = 520,000 \rightarrow 0.078 \text{ USD}$$

ประหยัดค่า input ได้ $(0.174 - 0.078)/0.174 \approx 55\%$

บทเรียนสองข้อ: (1) พรมป้ตระบบคือต้นทุนที่ถูกจ่ายซ้ำทุกคำขอ (2) เขียนคำสั่งเป็นภาษาอังกฤษแล้วให้ตอบภาษาไทย มักถูกกว่าและได้ผลดีกว่าด้วย

ขั้นที่ 2: เวกเตอร์ฝังตัว (Embeddings)

โทเคนแต่ละตัวถูกจับคู่กับเวกเตอร์ที่เรียนรู้ได้ในมิติ d (เช่น 4096)

$$E \in \mathbb{R}^{V \times d}, \quad \mathbf{e}_i = E[\text{token}_i]$$

ความหมายถูกเข้ารหัสเป็น **ทิศทาง** ในปริภูมิ โทเคนที่มีความหมายใกล้เคียงกันจะอยู่ใกล้กัน

$$\text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$$

ขนาดของตารางนี้ไม่เล็ก: คลังศัพท์ 128k โทเคน $\times$ 4096 มิติ $\times$ 2 ไบต์ $\approx$ 1 GB สำหรับตารางฝังตัวอย่างเดียว

โคไซน์ซิมิลาริตีตัวนี้จะกลับมาอีกครั้ง ในสัปดาห์ที่ 10 เมื่อเราทำ RAG เพราะการค้นคืนเอกสารก็คือการหาเวกเตอร์ที่ใกล้กันนั่นเอง

ปัญหาที่สอง: ลำดับมีความหมาย

"หมากัดคน" กับ "คนกัดหมา" มีโทเคนชุดเดียวกัน แต่ความหมายต่างกันสิ้นเชิง

สาเหตุเชิงคณิตศาสตร์: กลไกความสนใจเป็น **permutation equivariant** ถ้าสลับลำดับ โทเคนเข้า ผลลัพธ์ก็แค่สลับตามไป ด้วย โมเดลจึงไม่มีทางรู้ลำดับเลย ถ้าเราไม่ใส่ข้อมูลตำแหน่งเข้าไปเอง

จึงต้องเพิ่ม การเข้ารหัสตำแหน่ง (**positional encoding**)

- แบบดั้งเดิม: ฟังก์ชันไซน์และโคไซน์ที่ความถี่ต่างกัน บวกเข้ากับเวกเตอร์ฝังตัว
- แบบที่นิยมในปัจจุบัน: **RoPE (Rotary Position Embedding)** หมุนเวกเตอร์ **q** และ **k** ตามตำแหน่ง ทำให้ผลคูณจุดขึ้นกับระยะห่างสัมพัทธ์ ($m - n$) ไม่ใช่ตำแหน่งสัมบูรณ์

สรุปชั่วโมงที่ 1

ข้อความ --> โทเคน --> เวกเตอร์ฝังตัว --> + ตำแหน่ง --> พร้อมเข้าโมเดล
"AI" [15234] [0.2, -1.1, ...] RoPE

- โครงข่ายประสาทเรียนรู้ด้วย gradient descent + backpropagation ทั้งหมด
- โมเดลภาษาวัดความผิดพลาดด้วย cross-entropy รายงานเป็น perplexity
- **BPE** ค้นพบหน่วยย่อยของคำได้เอง โดยรวมคู่ที่พบบ่อยซ้ำ ๆ
- ภาษาไทยใช้โทเคนมากกว่าอังกฤษ 2 ถึง 4 เท่า ซึ่งแปลว่า **แพงกว่าและช้ากว่า**
- ความหมายคือ **ทิศทางในปริภูมิ** วัดด้วยโคไซน์
- ตำแหน่งต้องใส่เข้าไปเอง เพราะ attention ไม่รับรู้ลำดับโดยธรรมชาติ

พัก 10 นาที

ชั่วโมงที่ 2

กลไกความสนใจ

แนวคิด: ให้แต่ละโทเคน "มอง" โทเคนอื่น

เมื่ออ่านคำว่า "มัน" ในประโยค เราต้องรู้ว่ามันหมายถึงอะไร กลไกความสนใจให้โมเดลดึงข้อมูลจากโทเคนอื่นมาผสมเข้ากับตัวเอง

แต่ละโทเคนสร้างเวกเตอร์สามชุดจากเวกเตอร์ฝังตัวของตัวเอง

Query $q = W_Q x$

"ฉันกำลังมองหาอะไร"

Key $k = W_K x$

"ฉันมีอะไรให้"

Value $v = W_V x$

"เนื้อหาที่ฉันจะส่งให้"

เปรียบเทียบง่าย ๆ กับการค้นข้อมูล: query คือคำค้น key คือดัชนี value คือเนื้อหา ต่างกันตรงที่ที่นี้ "ค้นเจอ" ไม่ใช่ใช่หรือไม่ใช่ แต่เป็นน้ำหนักต่อเนื่องที่รวมกันได้ 1

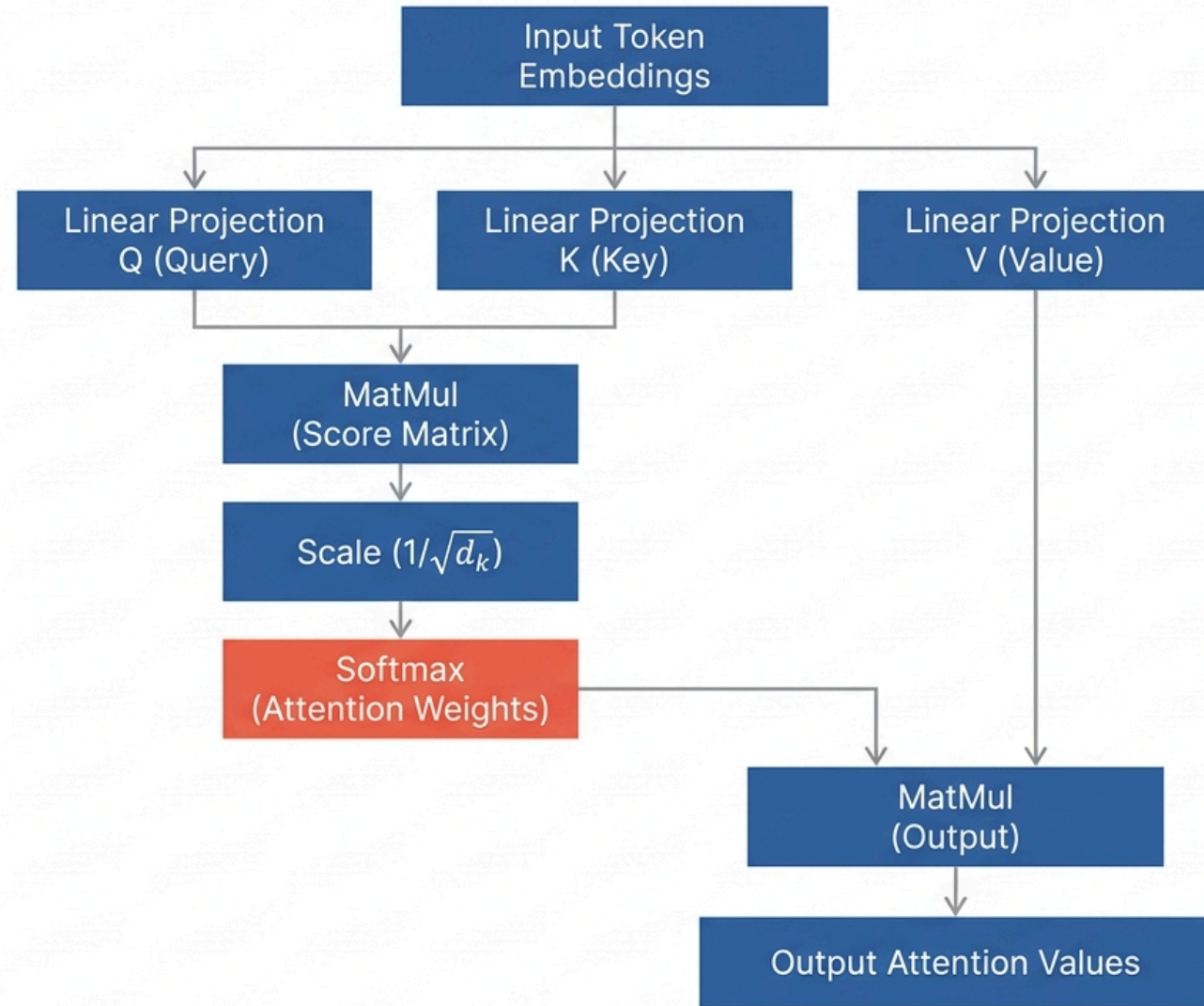
สมการเดียวที่ต้องจำ

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

อ่านจากในออกนอก:

1. $QK^\top$ คือคะแนนความเข้ากันของทุกคู่โทเคน (เมทริกซ์ $n \times n$)
- 2.หารด้วย $\sqrt{d_k}$ กันไม่ให้คะแนนใหญ่จนซอฟต์แวร์แมชชีนอึดตัว
- 3.ซอฟต์แวร์แมชชีนทำให้แต่ละแถวรวมกันได้ 1 กลายเป็นการแจกแจงความน่าจะเป็น
- 4.คูณด้วย V คือเฉลี่ยถ่วงน้ำหนักของเนื้อหาจากทุกโทเคน

ค่าใช้จ่ายเป็น $O(n^2)$ ตามความยาวบริบท n นี่เป็นเหตุผลที่บริบทยาวแพงมาก และเป็นแรงจูงใจของ RAG ในสัปดาห์ที่ 10



ทำไมต้องหารด้วย $\sqrt{d_k}$

สมมติ $\mathbf{q}$ และ $\mathbf{k}$ มีสมาชิกเป็นตัวเลขสุ่มอิสระ ค่าเฉลี่ย 0 ความแปรปรวน 1

$$\mathbf{q} \cdot \mathbf{k} = \sum_{i=1}^{d_k} q_i k_i \implies \text{Var}(\mathbf{q} \cdot \mathbf{k}) = d_k$$

ส่วนเบี่ยงเบนมาตรฐานจึงเป็น $\sqrt{d_k}$ ถ้า $d_k = 128$ คะแนนจะกระจายอยู่ในช่วง ± 11

ผลถ้าไม่หาร: ซอฟต์แมกซ์ของคะแนนที่ใหญ่มากจะเข้าใกล้ one-hot คือให้น้ำหนัก 1 กับตัวเดียวและ 0 กับที่เหลือ ทำให้ **ความ**
ชัน เกือบเป็นศูนย์ และโมเดลหยุดเรียนรู้

$$\text{softmax}([11, 0, -11]) \approx [0.99998, 0.0000167, 0.0000000003]$$

หารด้วย $\sqrt{d_k}$ แล้วความแปรปรวนกลับมาเป็น 1 พอดี

ตัวอย่างเต็มรูปแบบ: attention 3 โทเคน (ตั้งโจทย์)

ใช้ $d_k = 2$ เพื่อให้คำนวณด้วยมือได้ สมมติหลังการฉายเชิงเส้นได้

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \quad V = \begin{bmatrix} 10 & 0 \\ 0 & 10 \\ 5 & 5 \end{bmatrix}$$

แต่ละแถวคือหนึ่งโทเคน เรียกว่าโทเคน 1, 2, 3

จงหา ผลลัพธ์ของ โทเคนที่ 3 โดยไม่ใส่หน้ากาก

ขั้นตอนที่ต้องทำ: คำนวณ $\mathbf{q}_3 K^\top$, หารด้วย $\sqrt{2}$, ซอฟต์แมกซ์, คูณ V

เฉลย: ขั้นที่ 1 และ 2 (คะแนนดิบและการสเกล)

$q_3 = [1, 1]$ คูณกับทุกแถวของ K

$$q_3 \cdot k_1 = (1)(1) + (1)(0) = 1$$

$$q_3 \cdot k_2 = (1)(0) + (1)(1) = 1$$

$$q_3 \cdot k_3 = (1)(1) + (1)(1) = 2$$

คะแนนดิบ: $[1, 1, 2]$

สเกลด้วย $\sqrt{d_k} = \sqrt{2} \approx 1.414$

$$\left[\frac{1}{1.414}, \frac{1}{1.414}, \frac{2}{1.414} \right] = [0.707, 0.707, 1.414]$$

เฉลย: ขั้นที่ 3 (ซอฟต์แวร์แมกซ์)

$$e^{0.707} = 2.028, \quad e^{0.707} = 2.028, \quad e^{1.414} = 4.113$$

$$\text{ผลรวม} = 2.028 + 2.028 + 4.113 = 8.169$$

$$\alpha = \left[\frac{2.028}{8.169}, \frac{2.028}{8.169}, \frac{4.113}{8.169} \right] = [0.248, 0.248, 0.503]$$

ตรวจสอบ: $0.248 + 0.248 + 0.503 = 0.999 \approx 1$ ถูกต้อง

ตีความ: โทเคนที่ 3 ให้น้ำหนักกับตัวเองมากที่สุด (0.503) เพราะ q_3 กับ k_3 ชี้ไปทางเดียวกันที่สุด ส่วนโทเคน 1 และ 2 ให้น้ำหนักเท่ากันเพราะสมมาตรกัน

เฉลย: ขั้นที่ 4 (คูณด้วย V)

$$\begin{aligned}\text{output}_3 &= 0.248 [10, 0] + 0.248 [0, 10] + 0.503 [5, 5] \\ &= [2.48, 0] + [0, 2.48] + [2.515, 2.515] \\ &= [4.995, 4.995]\end{aligned}$$

สังเกต: ผลลัพธ์เป็นการผสมเนื้อหาจากทั้งสามโทเคน ไม่ใช่การเลือกอันใดอันหนึ่ง

ประเด็นที่ต้องเข้าใจให้ลึก: attention ไม่ได้เลือก โทเคน แต่ เฉลี่ยถ่วงน้ำหนัก ทุกโทเคนเสมอ ต่อบริบทจะน้อยแค่ไหนก็ยังมีส่วนร่วม นี่คือเหตุผลที่เอกสารที่ไม่เกี่ยวข้องในบริบทยัง รบกวน ผลลัพธ์ได้ ซึ่งเราจะเจออีกครั้งในสัปดาห์ที่ 9 และ 10

หน้ากากเชิงสาเหตุ (Causal Mask)

โมเดลภาษาต้องทำนายโทเคนถัดไป จึง **ห้ามมองอนาคต**

	t1	t2	t3	t4
t1	[ok	-inf	-inf	-inf]
t2	[ok	ok	-inf	-inf]
t3	[ok	ok	ok	-inf]
t4	[ok	ok	ok	ok]

ใส่ $-\infty$ ในตำแหน่งอนาคตก่อนซอฟต์แวร์แมชชีน ค่าเหล่านั้นจะกลายเป็น 0

ถ้าไม่มีหน้ากาก: โมเดลจะเห็นคำตอบตอนฝึก ค่าสูญเสียจะต่ำมากตั้งแต่รอบแรก แต่ตอนใช้งานจริงจะพังสิ้นเชิง เพราะอนาคตยังไม่มีอยู่ นี่คือรูปแบบหนึ่งของ **data leakage**

นี่คือความแตกต่างหลักระหว่างสถาปัตยกรรมแบบ **decoder-only** (GPT, Claude, Llama, Qwen) กับ **encoder** (BERT) ที่มองได้ทั้งสองทางและเหมาะกับงานจำแนกประเภทมากกว่า

แบบฝึกหัด 2.1: ใส่หน้ากากแล้วผลเปลี่ยนอย่างไร

ใช้โจทย์เดิม แต่คราวนี้คำนวณผลลัพธ์ของ **โทเคนที่ 2** พร้อมหน้ากากเชิงสาเหตุ

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \quad V = \begin{bmatrix} 10 & 0 \\ 0 & 10 \\ 5 & 5 \end{bmatrix}$$

คำใบ้: โทเคนที่ 2 มองเห็นได้เฉพาะโทเคน 1 และ 2 เท่านั้น

เฉลย 2.1

$q_2 = [0, 1]$ คะแนนดิบกับ key ที่มองเห็นได้

$$q_2 \cdot k_1 = 0, \quad q_2 \cdot k_2 = 1, \quad q_2 \cdot k_3 = 1 \rightarrow -\infty \text{ (ถูกปิด)}$$

สเกล: $[0, 0.707, -\infty]$

ซอฟต์แวร์แมกซ์: $e^0 = 1, e^{0.707} = 2.028, e^{-\infty} = 0$, ผลรวม = 3.028

$$\alpha = [0.330, 0.670, 0]$$

$$\text{output}_2 = 0.330 [10, 0] + 0.670 [0, 10] = [3.30, 6.70]$$

เปรียบเทียบ: ถ้าไม่มีหน้ากาก โทเคน 3 จะมีส่วนร่วมและดึงผลลัพธ์เข้าหา $[5, 5]$ การปิดอนาคตเปลี่ยนคำตอบจริง ๆ ไม่ใช่แค่พิธีกรรม

ความสนใจหลายหัว (Multi-Head Attention)

แทนที่จะทำความสนใจครั้งเดียวด้วยมิติ d เราแบ่งเป็น h หัว หัวละ d/h มิติ

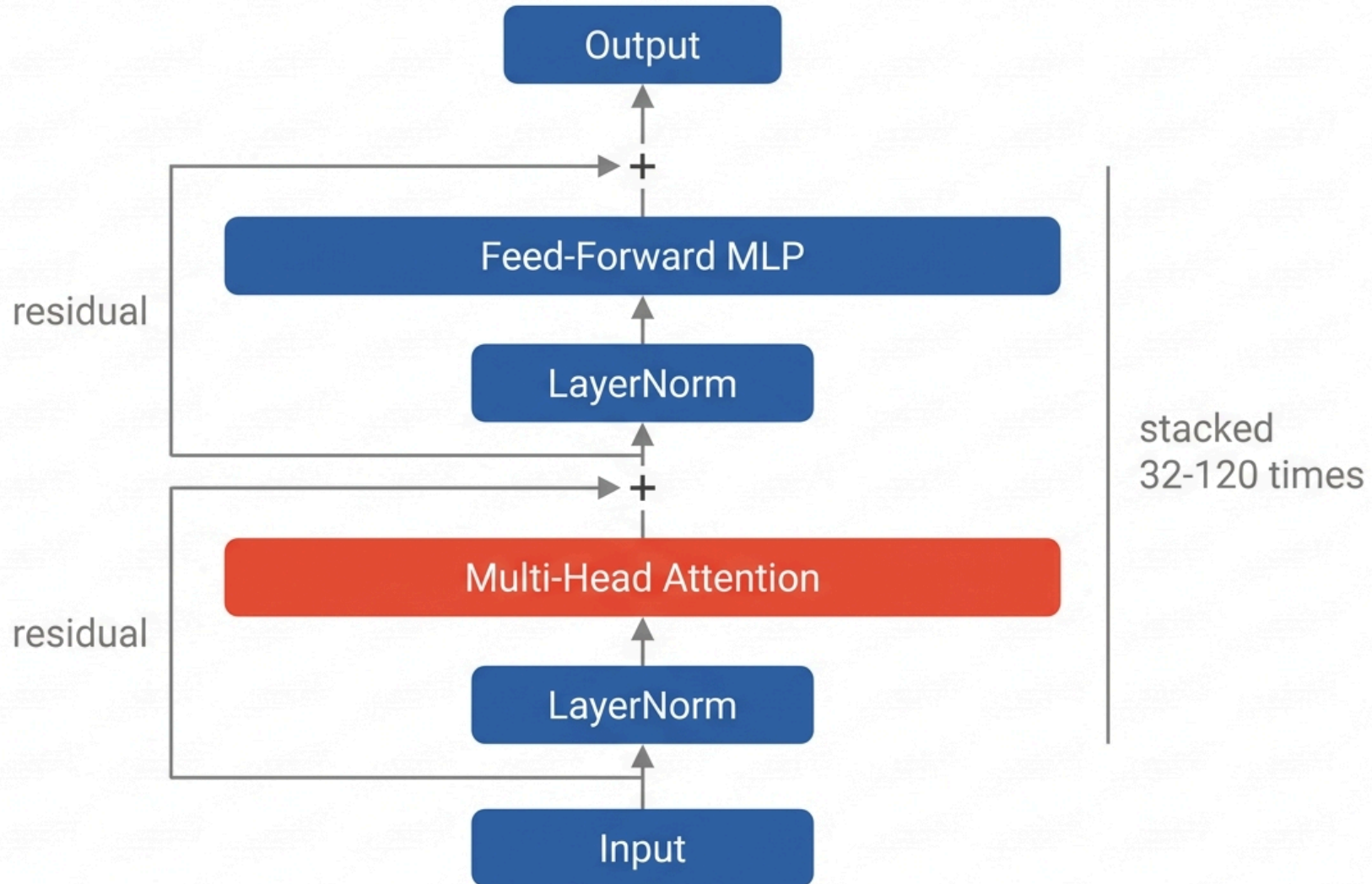
$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O$$

ทำไมแบ่งแล้วดีกว่า: ซอฟต์แวร์แมชชีนตัวเดียวถูกบังคับให้เลือก "รูปแบบความสัมพันธ์เดียว" ต่อหนึ่งโทเคน การแบ่งหัวให้โมเดลจับความสัมพันธ์หลายชนิดพร้อมกันได้ โดยค่าใช้จ่ายรวมเท่าเดิม

จากการวิเคราะห์โมเดลจริงพบว่า

- บางหัวตามหาคำสรรพนามอ้างอิง
- บางหัวจับคู่วงเล็บเปิดปิด
- บางหัวสนใจโทเคนก่อนหน้าทันที
- หลายหัวยังตีความไม่ได้

บล็อกทรานส์ฟอร์มเมอร์



ทำไมแต่ละชั้นในบล็อกถึงจำเป็น

ชั้นส่วน

ถ้าไม่มีจะเกิดอะไร

Residual connection ความชันหายไประหว่างทาง ฟังก์ชันโมเดลลึกเกิน 10 ชั้นไม่ได้

LayerNorm ค่าในแต่ละชั้นแกว่งจนฟังก์ชันไม่นิ่ง ต้องลด learning rate จนช้ามาก

Feed-Forward (MLP) attention เป็นการเฉลี่ยเชิงเส้นล้วน ๆ ไม่มีที่เก็บ "ความรู้"

Multi-Head Attention โทเคนแต่ละตัวมองไม่เห็นกัน กลายเป็นแค่ MLP รายโทเคน

Residual ทำงานอย่างไร: ถ้า $y = x + f(x)$ แล้ว

$$\frac{\partial y}{\partial x} = I + \frac{\partial f}{\partial x}$$

พจน์ I รับประกันว่าความชันมี "ทางด่วน" ไหลย้อนกลับได้เสมอ ต่อให้ $\frac{\partial f}{\partial x}$ จะเล็กแค่ไหน

พารามิเตอร์ส่วนใหญ่อยู่ใน Feed-Forward ไม่ใช่ attention ประมาณ 2 ใน 3 ของโมเดลทั้งตัว งานวิจัยชี้ว่า FFN ทำหน้าที่คล้ายหน่วยความจำแบบ key-value ที่เก็บข้อเท็จจริง

แบบฝึกหัด 2.2: ต้นทุนของบริบทยาว

โมเดลมี 32 ชั้น, 32 หัว, มิติ $d = 4096$, เก็บค่าแบบ 2 ไบต์ (fp16)

- ก) KV cache ใช้หน่วยความจำเท่าไรต่อโทเคน 1 ตัว
- ข) ถ้าบริบทยาว 100,000 โทเคน ต้องใช้ VRAM เท่าไรสำหรับ KV cache
- ค) ถ้าเพิ่มบริบทจาก 10k เป็น 100k เมทริกซ์ attention โตขึ้นกี่เท่า

สูตร: KV cache ต่อโทเคน = $2 \times (\text{ชั้น}) \times d \times (\text{ไบต์})$
(เลข 2 คือเก็บทั้ง K และ V)

เฉลย 2.2

ก) ต่อโทเคน

$$2 \times 32 \times 4096 \times 2 \text{ ไบต์} = 524,288 \text{ ไบต์} \approx \mathbf{0.5 \text{ MB}}$$

ข) บริบท 100k โทเคน

$$0.5 \text{ MB} \times 100,000 = 50,000 \text{ MB} \approx \mathbf{50 \text{ GB}}$$

มากกว่าความจุของ GPU ระดับ A100 80GB ไปแล้วครึ่งใบ สำหรับผู้ใช้คนเดียว

ค) เมทริกซ์ attention

$$\frac{100,000^2}{10,000^2} = \frac{10^{10}}{10^8} = \mathbf{100 \text{ เท่า}}$$

นี่คือคำตอบเชิงวิศวกรรมว่าทำไมต้องมี RAG บริบทยาวไม่ได้แค่ "แพ่งขึ้นหน่อย" แต่แพ่งขึ้นแบบกำลึงสอง และกิน VRAM แบบเชิงเส้นที่ชันมาก การค้นเฉพาะส่วนที่เกี่ยวข้องมาใส่ จึงถูกกว่าการยัดทุกอย่างเข้าไปมหาศาล

สรุปชั่วโมงที่ 2

- Attention คือ การเฉลี่ยถ่วงน้ำหนักที่เรียนรู้ได้ ระหว่างโทเคนทุกคู่
- $\text{softmax}(QK^\top / \sqrt{d_k})V$ ทุกสัญลักษณ์มีเหตุผลรองรับ
 - $\sqrt{d_k}$ กันซอฟต์แวร์แมกซ์อิมตัวและความชันหาย
 - หน้ากากเชิงสาเหตุกัน data leakage
- มันเฉลี่ยเสมอ ไม่ได้เลือก สิ่งที่ไม่เกี่ยวข้องในบริบทจึงรบกวนผลลัพธ์ได้
- หลายหัว ให้จับความสัมพันธ์หลายชนิดพร้อมกัน โดยต้นทุนเท่าเดิม
- Residual และ LayerNorm คือสิ่งที่ทำให้ฝึกโมเดลเล็ก ๆ ได้จริง
- ต้นทุน $O(n^2)$ และ KV cache คือข้อจำกัดทางกายภาพที่กำหนดทุกอย่างในอีก 6 สัปดาห์

พัก 10 นาที

ชั่วโมงที่ 3

จากทรานส์ฟอร์มเมอร์สู่ LLM

LLM คือเครื่องทำนายโทเคนถัดไป

หัวใจของ โมเดลภาษาคือการประมาณการแจกแจงเงื่อนไข

$$P(x_t | x_1, x_2, \dots, x_{t-1})$$

และความน่าจะเป็นของทั้งลำดับคือกฎลูกโซ่ที่เราเรียนในสัปดาห์ที่ 6

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t | x_{<t})$$

สังเกต: นี่คือการจำลองความน่าจะเป็นแบบเดียวกับที่เราศึกษาในบทเบย์ ต่างกันแค่เราใช้โครงข่ายประสาทประมาณการแจกแจงแทนตารางความน่าจะเป็น ซึ่งจำเป็น เพราะตารางสำหรับบริบท 100 โทเคนจะมีขนาด $|V|^{100}$ ช่องซึ่งเป็นไปไม่ได้

การสร้างข้อความคือการสุ่มจากการแจกแจงนี้ทีละโทเคน แล้วป้อนกลับเข้าไปใหม่ (autoregressive)

การสุ่มโทเคน: อุณหภูมิ

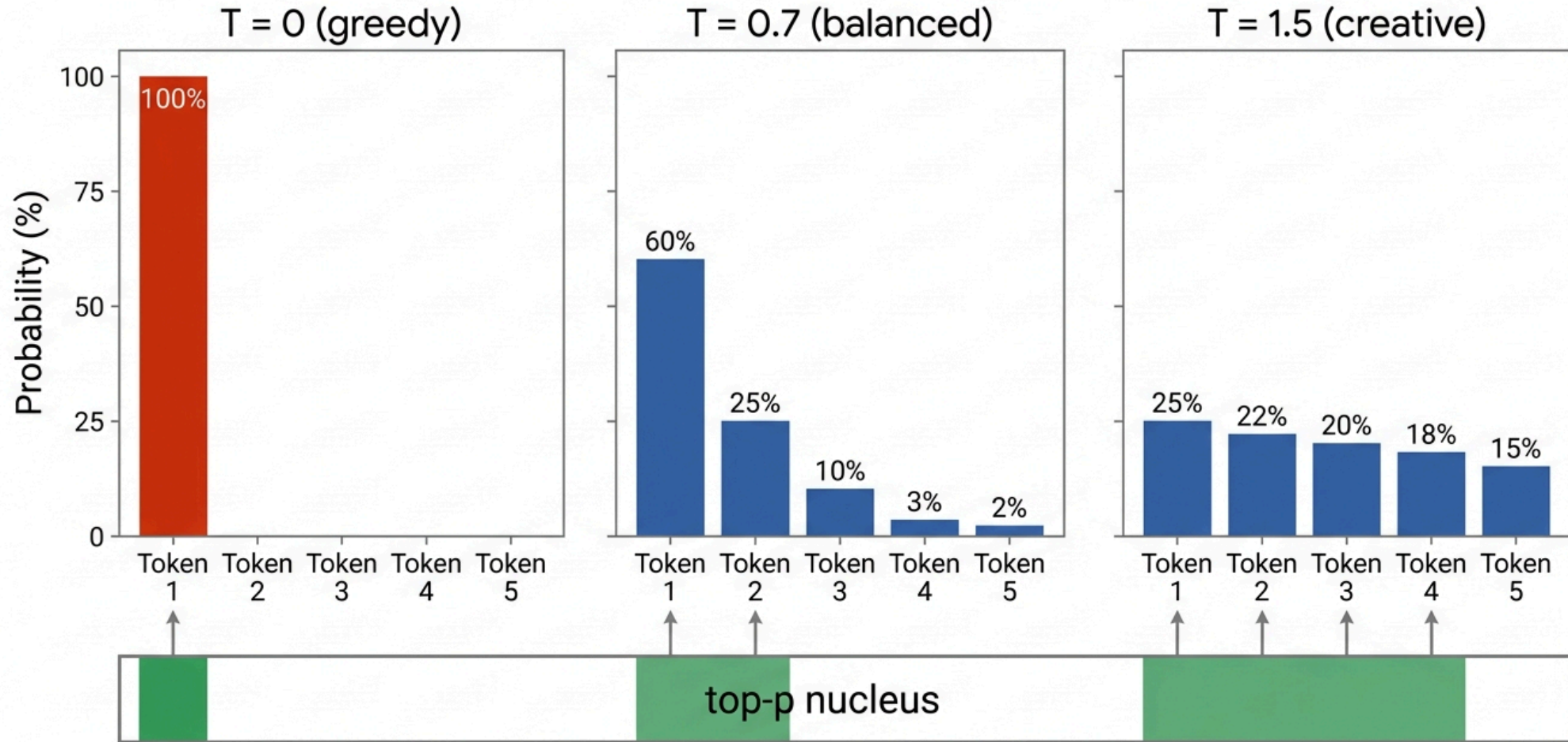
โมเดลให้ค่า logit z_i สำหรับทุก โทเคน ในคลังศัพท์ แล้วเราแปลงเป็นความน่าจะเป็น

$$P(i) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

T	ผลลัพธ์	เหมาะกับ
$T \rightarrow 0$	เลือกโทเคนที่น่าจะเป็นสูงสุดเสมอ (greedy)	สกัดข้อมูล, เขียนโค้ด, ตอบข้อเท็จจริง
$T = 0.7$	สมดุล	สนทนาทั่วไป
$T = 1.2$	หลากหลาย เสี่ยงเพี้ยน	งานสร้างสรรค์, ระดมความคิด

top-p (nucleus sampling): ตัดให้เหลือเฉพาะโทเคนที่ความน่าจะเป็นสะสมถึง p แล้วค่อยสุ่ม เป็นการกันหางยาวที่ไร้สาระออกไป

Temperature Reshapes Probability Distribution



แบบฝึกหัด 3.1: คำนวณผลของอุณหภูมิ

โมเดลให้ logits สำหรับ 3 โทเคนถัดไป

$$z = [\underbrace{2.0}_{\text{"แมว"}}, \underbrace{1.0}_{\text{"หมา"}}, \underbrace{0.0}_{\text{"นก"}}]$$

- ก) คำนวณความน่าจะเป็นที่ $T = 1.0$
- ข) คำนวณความน่าจะเป็นที่ $T = 0.5$
- ค) ถ้าใช้ top-p = 0.9 ที่ $T = 1.0$ โทเคนไหนถูกตัดออก

ค่าที่ใช้ได้: $e^2 = 7.39$, $e^1 = 2.72$, $e^0 = 1.00$, $e^4 = 54.6$, $e^2 = 7.39$

เฉลย 3.1 (ก และ ข)

ก) $T = 1.0$

$$\text{ผลรวม} = 7.39 + 2.72 + 1.00 = 11.11$$

$$P = \left[\frac{7.39}{11.11}, \frac{2.72}{11.11}, \frac{1.00}{11.11} \right] = [0.665, 0.245, 0.090]$$

ข) $T = 0.5$ ทาร logit ด้วย 0.5 ก่อน ได้ $z/T = [4.0, 2.0, 0.0]$

$$\text{ผลรวม} = 54.6 + 7.39 + 1.00 = 62.99$$

$$P = [0.867, 0.117, 0.016]$$

ตีความ: ลดอุณหภูมิลงครึ่งหนึ่ง ความน่าจะเป็นของตัวเต็งเพิ่มจาก 0.665 เป็น 0.867 และหางถูกกดจาก 0.090 เหลือ 0.016
การแจกแจง **แหลมขึ้น** แต่ลำดับไม่เปลี่ยน

เฉลย 3.1 (ค)

ที่ $T = 1.0$: $P = [0.665, 0.245, 0.090]$

เรียงจากมากไปน้อยแล้วสะสม

โทเคน P สะสม

แมว 0.665 0.665

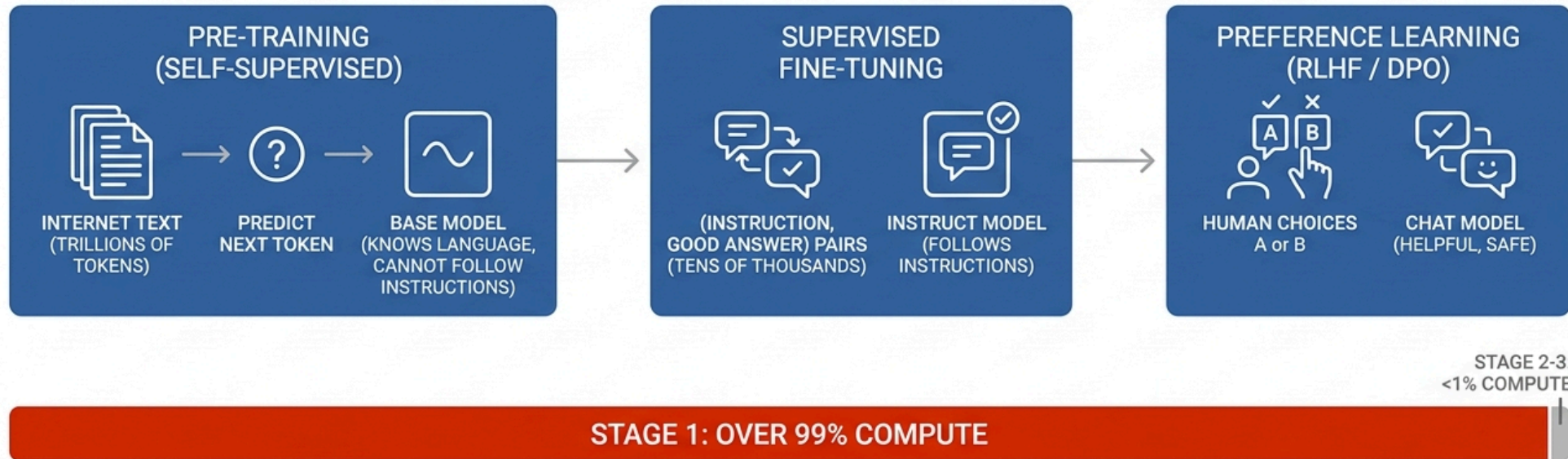
หมา 0.245 0.910

นก 0.090 1.000

$\text{top-p} = 0.9$ หมายถึงเก็บโทเคนจนความน่าจะเป็นสะสมถึง 0.9

- เก็บ "แมว" (0.665) ยังไม่ถึง 0.9
- เก็บ "หมา" ได้ 0.910 ถึงแล้ว หยุด
- "นก" ถูกตัดออก

วงการสร้างโมเดลสมัยใหม่



แต่ละขั้นให้อะไร

1. Pre-training

ข้อมูล: ข้อความหลายล้านล้านโทเคน
เป้าหมาย: ทำนายโทเคนถัดไป
ต้นทุน: **กว่า 99%** ของการคำนวณทั้งหมด

ได้: โมเดลที่รู้ภาษา รู้ข้อเท็จจริง แต่ **ทำตามคำสั่งไม่เป็น** ถามอะไรไปมันจะเดาว่าข้อความต่อไปคืออะไร ซึ่งอาจเป็นคำถามข้อถัดไป

2. SFT

ข้อมูล: คู่ (คำสั่ง, คำตอบที่ดี) หลักหมื่นถึงแสน เขียนโดยคน
เป้าหมาย: เลียนแบบคำตอบที่ดี

ได้: โมเดลที่ตอบคำถามเป็น รู้ว่าเมื่อถูกถามต้องตอบ ไม่ใช่ถามกลับ

3. RLHF / DPO

ข้อมูล: คนเลือกกว่าคำตอบ A หรือ B ดีกว่า
เป้าหมาย: เพิ่มความน่าจะเป็นของคำตอบที่คนชอบ

ได้: โมเดลที่มีประโยชน์ ปลอดภัย ไม่หยาบคาย และปฏิเสธคำขอที่ไม่เหมาะสม

สิ่งที่ต้องเข้าใจ: ขั้นที่ 3 สอน พฤติกรรม ไม่ได้สอน ความรู้ โมเดลไม่ได้ฉลาดขึ้นจาก RLHF แต่มัน ใช้งานง่ายขึ้น อย่างมหาศาล และนี่คือเหตุผลที่ RAG (ความรู้) กับ fine-tuning (พฤติกรรม) แก้ปัญหาคนละอย่างกัน

กฎการปรับขนาด (Scaling Laws)

ประสิทธิภาพดีขึ้นอย่างคาดเดาได้ตามกฎกำลัง เมื่อเพิ่ม พารามิเตอร์ N , ข้อมูล D , และ การคำนวณ C

$$L(N) \approx \left(\frac{N_c}{N}\right)^\alpha, \quad \alpha \approx 0.076$$

- Chinchilla (2022) แสดงว่า โมเดลยุคแรกใหญ่เกินไปเมื่อเทียบกับข้อมูล สัดส่วนที่เหมาะสมคือประมาณ 20 โทเคนต่อ 1 พารามิเตอร์
- ยุคปัจจุบันเพิ่มมิติที่สี่: การคำนวณตอนอนุมาน (inference-time compute) โมเดลแบบให้เหตุผลใช้โทเคนคิดในใจก่อนตอบ แลกเวลากับความแม่นยำ

ความสามารถที่ผุดขึ้น (emergent): งานบางอย่างเช่นการคำนวณหลายชั้น แทบทำไม่ได้เลยจนกว่า โมเดลจะถึงขนาดหนึ่ง แล้วจึงทำได้ทันที นี่ทำให้ทำนายความสามารถของรุ่นถัดไปได้ยาก

หน้าต่างบริบทและ KV cache

หน้าต่างบริบท (context window) คือจำนวนโทเคนสูงสุดที่โมเดลมองเห็นได้ในครั้งเดียว รวมทั้งคำสั่งระบบ ประวัติสนทนา เอกสารที่แนบ และคำตอบที่กำลังสร้าง

KV cache: ระหว่างสร้างข้อความ เราเก็บ key และ value ของโทเคนเดิมไว้ จึงไม่ต้องคำนวณซ้ำ ทำให้สร้างโทเคนถัดไปเร็วขึ้นมาก แต่กินหน่วยความจำ GPU เพิ่มตามความยาว อย่างที่เราคำนวณกันไปแล้ว ในแบบฝึกหัด 2.2 คือ 0.5 MB ต่อโทเคน

ผลที่ตามมาที่ต้องจำไว้ทั้งหมด: บริบทเป็นทรัพยากรจำกัดและมีราคา การจัดการว่าอะไรควรอยู่ในหน้าต่างบริบทเรียกว่า **context engineering** ซึ่งเป็นหัวข้อหลักของสัปดาห์หน้า

ข้อจำกัดที่ต้องเข้าใจ

- การหลอน (hallucination): โมเดลถูกฝึกให้สร้างข้อความที่ **น่าจะเป็น** ไม่ใช่ข้อความที่ **เป็นจริง** มันไม่มีกลไกแยกแยะสองอย่างนี้โดยกำเนิด
- วันตัดความรู้ (knowledge cutoff): ไม่รู้เหตุการณ์หลังวันที่ฝึก
- ไม่มีความจำถาวร: จบการสนทนาแล้วลืมหมด
- คณิตศาสตร์และตรรกะหลายชั้น: ยังผิดพลาดได้ง่าย
- ความอ่อนไหวต่อการใช้คำ: เปลี่ยนถ้อยคำเล็กน้อยอาจได้คำตอบต่างกันมาก

ข้อจำกัดสามข้อแรกคือเหตุผลที่มี RAG (สัปดาห์ 10), เครื่องมือ (สัปดาห์ 11 และ 12), และหน่วยความจำภายนอก ทุกหัวข้อที่เหลือของวิชานี้คือการแก้ข้อจำกัดเหล่านี้ด้วยวิศวกรรมรอบตัวโมเดล

ทำไมการหลอนถึงแก้ที่ตัวโมเดลไม่ได้

ลองคิดจากฟังก์ชันสูญเสีย โมเดลถูกฝึกให้ลด

$$-\log P(x_t^{\text{จริง}} | x_{<t})$$

ไม่มีที่ไหนในสมการนี้ที่พูดถึง "ความจริง" มันพูดถึงแค่ "ข้อความนี้ปรากฏในข้อมูลฝึกหรือไม่"

ผลที่ตามมาสามข้อ

1. ข้อความที่ฟังดูเหมือนจริงกับข้อความที่เป็นจริง ได้คะแนนใกล้เคียงกัน
2. โมเดลไม่มีตัวแทนภายในของ "ฉันไม่รู้" ที่แยกจาก "ฉันรู้" อย่างชัดเจน
3. การตอบว่า "ไม่ทราบ" พบน้อยในข้อมูลฝึก โมเดลจึงมี prior ที่จะตอบเสมอ

ทางแก้จึงอยู่นอกโมเดล: ให้เอกสารจริงไปด้วย (RAG), ให้เครื่องมือไปตรวจสอบ (สัปดาห์ 11 และ 12), บังคับให้อ้างอิงแหล่ง และให้รางวัลกับการตอบว่าไม่รู้ผ่านพรอมปต์

โมเดลที่ควรรู้จัก (พ.ศ. 2569)

แบบเรียกผ่าน API (closed weights)

- Claude (Anthropic)
- Gemini (Google)
- GPT (OpenAI)

ข้อดี: ความสามารถสูงสุด ไม่ต้องมี GPU ข้อเสีย: ข้อมูลออกนอกองค์กร มีค่าใช้จ่ายต่อโทเคน ถูกเลิกให้บริการได้

แบบเปิดน้ำหนัก (open weights)

- Qwen (Alibaba)
- GLM (Zhipu)
- Kimi (Moonshot)
- Llama, Mistral, Gemma, DeepSeek

ข้อดี: รันเองได้ ข้อมูลไม่ออกจากเครื่อง ปรับแต่งได้ ฟรี ข้อเสีย: ต้องมี GPU และดูแลเอง

ข่าวดี: อินเทอร์เฟซแทบจะเหมือนกันหมด

ผู้ให้บริการเกือบทั้งหมดรองรับรูปแบบ chat completions ที่เข้ากันได้กับ OpenAI

```
from openai import OpenAI

# เปลี่ยนแค่ base_url กับ model ก็ย้ายผู้ให้บริการได้
client = OpenAI(base_url="http://localhost:11434/v1", api_key="ollama")

resp = client.chat.completions.create(
    model="qwen3:8b",
    messages=[
        {"role": "system", "content": "คุณเป็นผู้ช่วยสอนวิชา AI"},
        {"role": "user", "content": "อธิบาย attention ใน 2 ประโยค"},
    ],
    temperature=0.3,
)
print(resp.choices[0].message.content)
```

หลักการของวิชานี้: เราจะเขียนโค้ดให้ไม่ผูกกับผู้ให้บริการรายใดรายหนึ่ง สิ่งที่คุณเรียนต้องใช้ได้กับทุกโมเดล

ช่วงลงมือทำ: labs/w08_llm_internals.ipynb

เปิดโน้ตบุ๊กแล้วทำไปพร้อมกัน ทุกเซลล์รันได้โดยไม่ต้องมี API key

1. วัดต้นทุนภาษาไทย

```
for name, s in [("ไทย", TH), ("อังกฤษ", EN)]:  
    print(len(s), len(s.encode()))
```

ยืนยันตัวเลข 3 ไบต์ต่ออักษรที่เราคุยกันในชั่วโมงที่ 1

2. BPE ด้วยโค้ด

รัน train_bpe แล้วเทียบกับที่เราทำด้วยมือในแบบฝึกหัด 1.1 merge ที่ได้ต้องตรงกัน

3. Attention จากศูนย์

```
_, A = attention(X, X, X, mask=mask)  
assert np.allclose(np.triu(A, 1), 0)
```

ตรวจว่าหน้ากากทำงาน และแต่ละแถวรวมได้ 1

4. อุณหภูมิ

รัน sample() ที่ T ต่าง ๆ แล้วดูการกระจายของผลลัพธ์ 20 ครั้ง เทียบกับที่เราคำนวณในแบบฝึกหัด 3.1

จุดที่ต้องหยุดคิด: เซลล์ที่ 4 มี `assert sample(logits, 0.0) == 0` ลองแก้ไข T เป็น 0.01 แทน แล้วดูว่ายังผ่านไหม เพราะอะไร

สรุป

- LLM คือทรานส์ฟอร์เมอร์แบบ decoder-only ที่ฝึกให้ทำนายโทเคนถัดไป
- **Attention** $\text{softmax}(QK^T/\sqrt{d_k})V$ ให้แต่ละโทเคน **เฉลี่ยถ่วงน้ำหนัก** ข้อมูลจากโทเคนอื่น ด้วยต้นทุน $O(n^2)$ ตามความยาวบริบท
- ความสามารถมาจาก **การปรับขนาด** ส่วนความมีประโยชน์มาจาก **SFT และ RLHF**
- **อุณหภูมิและ top-p** ควบคุมความสุ่มของผลลัพธ์ ปรับตามงาน
- การหลอนเป็นผลโดยตรงจากฟังก์ชันสูญเสีย จึง **แก้ที่ตัวโมเดลไม่ได้**

สัปดาห์หน้า: เราจะเลิกมองมันเป็นทฤษฎี แล้วเริ่มเรียกใช้จริงผ่าน API พร้อมเรียนรู้ว่าจะจัดการหน้าต่างบริบทอย่างไรให้ได้ผลลัพธ์ที่เชื่อถือได้

การบ้านสัปดาห์ที่ 8

ส่วนที่ 1: คำนวณด้วยมือ (ไม่ต้องใช้คอมพิวเตอร์)

1. ทำแบบฝึกหัด 1.1 ต่ออีก 3 merge แล้วบอกว่า BPE ค้นพบหน่วยคำอะไรได้บ้าง
2. คำนวณ attention ของโทเคนที่ 1 จากโจทย์ในชั่วโมงที่ 2 พร้อมหน้ากาก
3. โมเดลหนึ่งมี 80 ชั้น มิติ 8192 เก็บแบบ fp16 ถ้ามีผู้ใช้พร้อมกัน 20 คน คนละบริบท 32k โทเคน ต้องใช้ VRAM สำหรับ KV cache รวมเท่าไร

ส่วนที่ 2: แล็บ labs/w08_llm_internals.ipynb

1. รันครบทุกเซลล์ ทำ TODO ข้อ 1 (positional encoding) และข้อ 3 (เทียบกับ PyTorch)
2. ตอบคำถาม: คุณหภูมิเปลี่ยนผลลัพธ์อย่างไร และคุณจะเลือกค่าเท่าไรสำหรับงานสกัดข้อมูล เพราะอะไร

